Hospital Emergency Resource Allocation System Using OpenVINO: A Machine Learning-Based Approach to Inference Optimization and Resource Planning

Radhika M
School of Computer Science and Engineering,
Vellore Institute of Technology,
Vellore, India
radhika.2026@vitstudent.ac.in

Pavithra S
School of Computer Science and Engineering,
Vellore Institute of Technology,
Vellore, India
pavithra.2026@vitstudent.ac.in

***Abstract*—Hospital emergency departments (EDs) frequently experience unpredictable patient arrivals, overcrowding, and inefficient utilization of limited resources such as emergency beds, medical staff, patient monitors, and oxygen units. This paper presents a prototype Hospital Emergency Resource Allocation System that combines machine-learning-based patient acuity classification, Intel OpenVINO inference optimization, and hospital resource inventory monitoring. Emergency department data containing patient demographics, triage information, vital signs, diagnoses, and visit-disposition details were inspected and preprocessed, and 207 visits with recorded acuity labels were retained for modelling. A class-weighted Random Forest was evaluated as a baseline, followed by a Logistic Regression classifier that was exported to the Open Neural Network Exchange (ONNX) format and executed with OpenVINO. Random Forest achieved 46.15% accuracy and 39.34% balanced accuracy, while Logistic Regression achieved 51.92% accuracy. OpenVINO produced predictions identical to those of the original scikit-learn classifier for all 207 labelled records. Classifier-only benchmarks recorded speed-ups of approximately 1.72× to 2.07×, whereas end-to-end measurements that include preprocessing showed that this gain can be reduced or eliminated. On the tested hardware, CPU inference was approximately 3.22 times faster than GPU inference for this small model. The results are preliminary because the labelled dataset is small and imbalanced, and the system is intended to support operational planning rather than replace clinical judgement.**

Keywords—Emergency department, resource allocation, machine learning, Logistic Regression, Random Forest, OpenVINO, ONNX, inference latency

I. Introduction

Emergency departments (EDs) are critical components of healthcare systems because they provide immediate medical attention to patients with varying levels of urgency. Effective ED management requires timely patient assessment, appropriate prioritization, and efficient utilization of the available resources. However, unpredictable patient arrivals, overcrowding, and limited staff and equipment create persistent operational challenges.

Machine learning (ML) provides methods for identifying patterns in historical healthcare data and for supporting predictions about patient outcomes, workload, and ED demand. Zlotnik et al. investigated ML techniques for ED visit forecasting and dynamic nursing staff allocation, demonstrating the relevance of predictive analytics to workforce planning [1]. Similarly, Mizan and Taghipour integrated ML prediction with mathematical optimization for medical resource allocation [2], and El Ariss et al. developed an ML framework that predicts a broad range of resource requirements from ED triage data [3].

Although predictive models can provide useful information, their practical deployment also depends on computational efficiency. Intel OpenVINO is an inference toolkit that supports optimized execution of compatible trained models on supported hardware [4]. A trained model can be converted into an interoperable representation such as ONNX [5] and evaluated to determine whether inference optimization improves execution performance without changing the model's predictions.

The proposed Hospital Emergency Resource Allocation System Using OpenVINO combines ED data analysis, ML classification, optimized inference, and basic resource inventory tracking. The implementation evaluates Random Forest and Logistic Regression classifiers and deploys the Logistic Regression model through ONNX and OpenVINO. Its performance is examined using prediction agreement, classifier-only latency, end-to-end latency, and CPU versus GPU execution.

The problem addressed in this work can be stated as follows. EDs must make operational decisions under changing patient demand and limited resource availability, and inefficient resource planning can contribute to delays and overcrowding. This work investigates whether ML classification and optimized inference can support ED decision-making, while measuring the trade-offs between prediction quality, computational cost, and execution time. The current implementation focuses on model training, model conversion, prediction agreement, performance benchmarking, and a basic resource inventory; a fully validated, automated clinical resource-allocation system has not yet been demonstrated.

The main contributions of this paper are as follows:

- A prototype pipeline that integrates ED data inspection, preprocessing, acuity classification, ONNX conversion, OpenVINO inference, and a resource inventory dashboard.

- A comparison of Random Forest and Logistic Regression using accuracy and balanced accuracy on an imbalanced acuity dataset.
- A prediction-agreement check confirming that the OpenVINO-executed model reproduces the predictions of the original scikit-learn model.
- A benchmark that separates classifier-only latency from end-to-end pipeline latency and compares CPU and GPU execution.

The system is intended to support, rather than replace, decisions made by qualified healthcare professionals.

II. Motivation and Feasibility

A. Technical Feasibility

The project is technically feasible because publicly available healthcare datasets can be used for model development, and algorithms such as Random Forest, XGBoost, LightGBM, Logistic Regression, and neural networks have already demonstrated usefulness in ED prediction tasks. OpenVINO provides tools for converting, optimizing, and deploying trained models, together with benchmarking facilities for measuring inference latency and throughput [4]. The system is implemented using Python, Pandas, NumPy, scikit-learn [23], PyTorch, OpenVINO, and Streamlit. The prototype successfully executes the converted Logistic Regression classifier through OpenVINO and achieved 100% prediction agreement with the original classifier on 207 labelled records.

B. Economic Feasibility

The prototype uses freely available software tools and libraries and does not require a dedicated commercial AI server; development and initial testing can be performed on an existing computer. If optimized inference operates efficiently on existing hardware, it may reduce the need for additional computational infrastructure. However, the current experiments do not quantify financial savings or deployment cost, which would require a separate cost analysis.

C. Social Feasibility

ED overcrowding and inefficient resource utilization affect patients, clinicians, and hospital administrators. A reliable decision-support system could help staff understand patient acuity patterns and resource availability. The current prototype is not clinically validated and must not make autonomous treatment or triage decisions; its outputs should be evaluated for reliability and potential bias before any clinical use.

D. Environmental Feasibility

Inference optimization and local execution may reduce unnecessary transmission of patient information to remote services, and efficient use of existing hardware could reduce the need for additional computing resources. These environmental benefits have not been measured in this work and would require measurements of energy consumption and hardware utilization.

E. Political and Regulatory Feasibility

Healthcare software must respect applicable patient privacy, data-protection, cybersecurity, and medical-device requirements. The prototype should use appropriately authorized, de-identified data and restrict access to sensitive information. Before real-world deployment, the system would require institutional approval, security assessment, clinical evaluation, and compliance review; OpenVINO execution alone does not establish regulatory approval or clinical safety.

F. Demographic Feasibility

EDs serve patients from different age groups and demographic backgrounds, and characteristics such as age and sex may influence the distribution of presentations and model predictions. The dataset should therefore be examined for demographic representation and missingness, and performance should be evaluated across subgroups where sample sizes permit. The current small labelled sample is insufficient to establish fairness or generalizability across populations.

III. Literature Review

This section reviews twenty studies on ED demand forecasting, crowding prediction, triage-based outcome and resource prediction, and resource-allocation optimization, which together motivate the proposed system.

Xu et al. [6] modelled daily patient arrivals at an ED in order to support resource planning. They applied an artificial neural network (ANN) and compared it with nonlinear least-squares regression and multiple linear regression using one year of arrival data from a local ED. The ANN predicted arrivals more accurately than the regression models and was also used to quantify the relative importance of contributing variables, with temporal and influenza-related factors emerging as important predictors. The main limitations are the use of a single ED and a short data period, and the authors suggest testing on multiple hospitals and incorporating real-time data.

Zlotnik et al. [1] forecast ED visits and used the forecasts to allocate nursing staff dynamically. Using data from a 1100-bed specialized-care hospital involving approximately 553,000 patients, they compared support vector regression (SVR) and the M5P model tree, which fits linear regression models at its leaf nodes, against a baseline. SVR and M5P outperformed the baseline, and dynamic allocation was shown to substantially reduce understaffing. The study directly connects forecasting with staffing decisions, but it focuses only on the nursing workforce; integrating additional resources and modern deep learning methods is identified as future work.

Chen and Wang [7] addressed the allocation of doctors, nurses, laboratory staff, and equipment in an ED under capacity constraints. They combined discrete-event simulation of ED operations in Taiwan with multi-objective optimization, using the non-dominated sorting particle swarm optimization (NSPSO) algorithm and multi-objective computing budget allocation (MOCBA). The approach generated optimal allocation solutions that balance patient length of stay against resource cost and naturally handles multiple objectives and constraints. Its dependence on simulation makes it computationally expensive, and combining simulation optimization with ML-based demand prediction is proposed as future work.

Yousefi et al. [8] aimed to reduce ED overcrowding and length of stay through optimum resource allocation. Their method combines agent-based simulation of a public ED with an ensemble metamodel of an adaptive neuro-fuzzy inference system (ANFIS), a feed-forward neural network, and a recurrent neural network (RNN) built using AdaBoost, and a chaotic genetic algorithm then searches for the best resource configuration. The approach reduced the average

length of stay by 14%, and the ensemble metamodel improved the mean absolute percentage error (MAPE) by 26.6% over the individual models. The method is computationally intensive and relies on simulation assumptions, and faster real-time optimization using live hospital data is suggested.

Hong et al. [9] predicted hospital admission at the time of ED triage. Using adult ED visits from one academic and two community EDs between 2014 and 2017, they compared logistic regression, XGBoost, and deep neural networks. XGBoost achieved an area under the curve (AUC) of 0.91 when triage information was combined with patient history, showing high predictive performance that is useful for bed planning. Because the data originated from a single healthcare system, external validation and integration with bed and resource allocation remain open.

Raita et al. [10] predicted critical care and hospitalization outcomes at ED triage using 135,470 adult ED visits. They compared Lasso regression, random forest, gradient-boosted decision trees, and a deep neural network (DNN) with the conventional Emergency Severity Index (ESI). The DNN achieved an AUC of 0.86 for critical care and 0.82 for hospitalization, outperforming conventional triage. The study benefits from a large dataset and a direct comparison with standard triage, but it requires external, prospective validation before clinical deployment.

Harrou et al. [11] proposed a deep learning framework to forecast ED overcrowding and patient flow. Using ED data from the regional hospital centre in Lille, France, they compared a variational autoencoder (VAE) with RNN, LSTM, bidirectional LSTM, convolutional LSTM, gated recurrent unit, and convolutional neural network models. The VAE showed superior forecasting performance among the eight deep learning models. The work compares several architectures in depth, but it is limited to a single healthcare setting, and multicentre validation with real-time deployment is suggested.

Sterling et al. [12] predicted the number of resources a patient would require during an ED visit at the time of triage. They combined natural language processing (NLP) of nursing triage notes with clinical variables in a neural network, using 226,317 encounters from three academic EDs. The model achieved an accuracy of 66.5% and a macro-F1 score of 0.601 across resource categories, which was comparable to human raters. The study predicts resource requirements directly, but it was less accurate for high-resource patients, and the authors recommend improved NLP and prospective deployment.

Benevento et al. [13] predicted ED waiting times in order to support operational decisions and overcrowding management. Using two real ED datasets with 40,504 and 60,230 presentations, they compared random forest, SVR, ANN, and ensemble methods. The ensemble method was the most effective, while random forest provided a good trade-off between accuracy and computation. The work uses real-world data and supports real-time monitoring, but waiting-time prediction does not by itself allocate resources; integration with automatic resource allocation is a natural extension.

Joseph et al. [14] predicted the workload generated by individual ED patients, measured in work relative value units (wRVUs), to support load balancing between physicians. Using 204,064 adult visits to an urban academic ED, they compared linear regression, gradient-boosted trees, and neural networks. A neural network using free-text triage information achieved the lowest mean absolute error of 0.78 wRVUs. The study measures workload at the patient level, but it is single-centre and wRVUs are an indirect measure of workload; multicentre validation and real-time staff balancing are proposed.

Smith et al. [15] evaluated whether ED crowding prediction models remain reliable under temporal and geographic change. They trained random forest models on four datasets from an academic and a rural hospital covering pre-COVID and COVID-19 periods. Models were not substantially affected by temporal drift, but performance degraded under spatial drift, for example when models trained at one hospital were applied to another. The study highlights robustness and model drift as practical concerns and recommends continuous monitoring, recalibration, and external validation.

Mizan and Taghipour [2] allocated medical resources while reducing workload and patient waiting time by integrating ML-based forecasting with a mathematical optimization model. Using patient arrival and radiology workload data, the approach reduced radiologist workload by 25% and patient waiting time by 8.17%. The work is a strong example of combining prediction with optimization, but it focuses mainly on radiology resources; extension to multiple hospital resources and real-time allocation remains future work.

Vollmer et al. [16] developed a framework to forecast ED demand for operational planning using eight years of electronic attendance data from two London hospitals. They compared and combined time-series models, ML algorithms, and stacked ensembles. One-day-ahead forecasts achieved a MAPE of approximately 6.8% and 8.6% for the two hospitals, and simpler linear models sometimes outperformed more complex ML models. The study shows the value of combining traditional and ML approaches, and applying the framework to more hospitals and operational indicators is suggested.

Rocha and Rodrigues [17] forecast hourly, four-hour, eight-hour, and daily ED admissions using ten years (2009–2018) of admission data from a Portuguese hospital. They compared exponential smoothing, SARIMA, autoregressive and recurrent neural networks, XGBoost, and ensemble learning. The recurrent neural networks and XGBoost produced the most accurate hourly forecasts, and XGBoost was by far the most computationally efficient. The study compares several models across multiple horizons, but it uses a single hospital, which limits generalization.

Tuominen et al. [18] forecast ED occupancy up to 24 hours ahead using electronic health record data together with external variables such as hospital bed availability, traffic, and weather. They evaluated advanced models including DeepAR, N-BEATS, the Temporal Fusion Transformer, and LightGBM, and the advanced ML models outperformed traditional benchmarks by up to 15%. The models exploit many external variables, but they are more complex and require greater computational resources, and real-time deployment with integration into allocation systems is suggested.

El Ariss et al. [3] predicted individual patient resource requirements during ED triage, which is the study most closely related to the proposed project. Using 391,472 ED visits from the MIMIC-IV-ED database, they trained 144 voting-ensemble models with Azure AutoML, one for each

resource target, obtaining an average AUC of 0.82 and an accuracy of 0.76. The framework predicts many resource types, but it is based on retrospective data and requires external validation, larger datasets, and progress toward real-time deployment.

Porto [19] systematically reviewed ML and NLP approaches for improving ED triage, analysing 60 studies retrieved from PubMed, Scopus, Web of Science, IEEE Xplore, and the ACM Digital Library. Logistic regression was the most frequently used model, while XGBoost, gradient boosting, and DNNs showed the strongest performance, and common predictors included demographics and vital signs. The review is broad and identifies common methods, but it also reports a high risk of bias in many models and limited use of explainable AI (XAI), recommending more explainable, unbiased, and externally validated models.

Porto and Fogliatto [20] improved ED arrival forecasting to support staff and resource allocation using data from eleven EDs in three countries. They tested XGBoost, LightGBM, SVM with a radial basis function kernel, neural network autoregression, random forest, and an elastic-net linear model, combined with a feature-engineering approach. XGBoost was the best-performing model, with MAPE values ranging from 5.03% to 14.1% across datasets, and feature engineering improved all algorithms. The multicentre, multinational dataset is a strength, but the work predicts arrivals rather than actual resource allocation.

Blanco et al. [21] conducted a PRISMA-guided systematic review of AI methods for hospital ED demand forecasting, covering eleven eligible studies published between 2019 and July 2025. ML and deep learning methods generally outperformed classical time-series models, and external variables such as weather and calendar events consistently improved performance. External validation and explainability remained limited, and the review recommends multicentre validation, explainable AI, and hybrid forecasting approaches.

Schippers et al. [22] developed a real-time AI model to predict imminent ED crowding using an LSTM neural network trained on single-centre electronic health record and workforce data. For one-hour-ahead prediction the model achieved an AUC of 0.84, compared with 0.73 for the modified National Emergency Department Overcrowding Score (NEDOCS). The model supports real-time ED workflow, but it is a single-centre retrospective study that requires multicentre and prospective validation.

In summary, the reviewed literature demonstrates that ML can forecast ED demand, crowding, waiting times, admissions, and resource needs with useful accuracy, and that optimization can convert predictions into allocation decisions [2], [7], [8]. However, most studies evaluate prediction accuracy alone and rarely examine the deployment efficiency of the trained model, the consistency of predictions after model conversion, or the latency of the complete inference pipeline. These observations motivate the research gaps discussed next.

IV. Research Gaps

A. Prediction Is Often Separated from Allocation

Existing research frequently focuses either on predicting ED demand and allocating resources or on optimizing the execution of trained ML models, and these objectives are rarely evaluated together in a single workflow. This project combines ED data processing, ML classification, OpenVINO inference, and a resource inventory demonstration within one prototype.

B. Limited Evaluation of End-to-End Inference Performance

A model that executes faster in isolation does not necessarily make an entire application faster, because preprocessing, input conversion, model execution, and Python-level overhead all affect total prediction time. This project therefore evaluates both classifier-only latency and end-to-end latency.

C. Prediction Consistency After Model Conversion

Speed measurements alone are insufficient to establish that a converted model preserves the original model's predictions. The trained Logistic Regression classifier is converted to ONNX, executed through OpenVINO, and checked for prediction agreement; this demonstrates consistency on the tested records but does not by itself establish clinical accuracy.

D. Single-Resource Focus

Several studies concentrate on a single resource such as nurses, beds, or radiologists [1], [2]. The proposed system is designed to consider multiple resource categories simultaneously, including doctors, nurses, emergency beds, laboratory and imaging resources, ICU or critical-care capacity, and other treatment resources.

E. Limited Explainability

Hospital decision-makers need to understand why a system predicts high resource demand, and reviews report limited use of XAI [19], [21]. The proposed system should therefore include explainability methods such as feature importance or SHAP where applicable.

F. Generalizability

Recent reviews highlight insufficient external validation and limited generalizability across hospitals [21]. This can be addressed experimentally by comparing performance across temporal splits or available hospital subsets.

G. Trade-off Between Accuracy and Inference Speed

A highly accurate model may be computationally expensive. The proposed system evaluates prediction quality and inference performance together, which makes OpenVINO benchmarking an important component of the project.

V. Proposed System

A. System Architecture

The architecture of the proposed system is shown in Fig. 1. It consists of six stages: (1) patient input data, including demographics, vital signs, triage acuity, and arrival details; (2) data preprocessing; (3) an ML prediction model that classifies patient acuity; (4) an OpenVINO inference layer that executes the converted model on supported devices; (5) a resource allocation module that matches estimated requirements with available beds, medical staff, monitors, and oxygen units; and (6) a hospital dashboard implemented in Streamlit. A feedback loop updates resource availability, incorporates new patient data, and re-runs inference periodically. The overall workflow is: ED dataset → data inspection and preprocessing → model training and evaluation → ONNX conversion → OpenVINO inference → performance benchmarking → resource inventory demonstration.

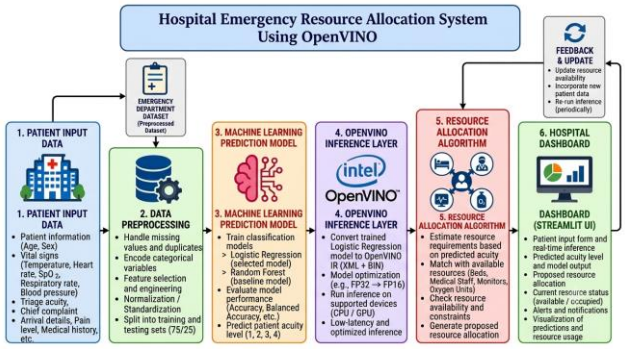


Fig. 1. Block diagram of the proposed Hospital Emergency Resource Allocation System using OpenVINO.

B. Dataset Description

MIMIC-IV-ED [24] is a particularly suitable dataset for this work because it contains ED data and has been used in recent research on emergency resource prediction; El Ariss et al. used 391,472 MIMIC-IV-ED visits to predict 144 resource-related targets [3]. Information available for modelling includes patient demographics, arrival information, triage information, vital signs, medications, diagnoses, procedures, disposition, and timing and ED-stay information.

The dataset used in the current experiments is organized as separate tables containing triage information, vital signs, diagnoses, medication information, and visit disposition, with 9,216 records in total. Inspection identified 222 visit IDs with varying overlap across tables, so record relationships were verified before joining to avoid duplicating visits. For the acuity prediction experiment, 207 visits with recorded acuity labels were retained; the class counts were 18, 97, 90, and 2 for acuity levels 1–4, respectively, which indicates strong class imbalance. The variable groups used are summarized in Table I. The percentage of missing values for each feature was computed as

$$P_{\text{missing}} = \frac{N_{\text{missing}}}{N_{\text{total}}} \times 100 \quad (1)$$

where N_{missing} is the number of missing values and N_{total} is the total number of values of the feature. This calculation identifies incomplete features and determines whether cleaning or imputation is necessary.

TABLE I. DATASET VARIABLE GROUPS

Group	Variables	Purpose
Patient information	Age, sex	Describe patient demographics
Vital signs	Temperature, heart rate, respiratory rate, SpO_2, blood pressure	Represent physiological condition
Triage information	Triage acuity	Target variable (acuity class)
Visit information	Visit ID, visit disposition	Identify visits and outcomes
Clinical information	Diagnosis, diagnosis code	Represent recorded conditions

C. Data Preprocessing

Raw healthcare data cannot always be used directly by an ML model. Preprocessing identified missing and duplicate records, selected relevant patient and visit features, prepared numerical features such as temperature, heart rate, respiratory rate, and oxygen saturation, encoded categorical features, and prepared the triage-acuity target. A scikit-learn pipeline applied median imputation and scaling to numerical features, and most-frequent imputation with one-hot encoding to categorical features, so that identical transformations were applied to training, testing, and inference inputs. After encoding, each record is represented by a 168-dimensional feature vector. When min–max normalization is applied, a feature value is transformed as

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (2)$$

where x is the original value and $x_{\min}$ and $x_{\max}$ are the minimum and maximum values of the feature. This scales values to the range [0, 1]. When standardization is applied, the transformation is

$$z = \frac{x - \mu}{\sigma} \quad (3)$$

where μ is the mean and σ is the standard deviation of the feature. Standardization helps models such as Logistic Regression when input features have different numerical scales. The data were then divided into training (75%) and testing (25%) subsets.

D. Classification Models

Random Forest [25] is an ensemble learning algorithm that combines the predictions of multiple decision trees. For classification, the predicted class is determined by majority voting among T trees:

$$\hat{y} = mode(h_1(x), h_2(x), \ldots, h_T(x)) \quad (4)$$

A class-weighted Random Forest was used as the baseline to partially compensate for the imbalanced class distribution. Logistic Regression is a supervised algorithm that estimates the probability of a class from the input features. For binary classification, the sigmoid function is

$$P(y = 1 \mid x) = \frac{1}{1+e^{-z}},\ z = w^T x + b \quad (5)$$

where w is the weight vector and b is the bias. For the multiclass acuity problem, the softmax function assigns a probability to each of the K classes:

$$P(y = k \mid x) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}} \quad (6)$$

E. ONNX Conversion and OpenVINO Inference

After training, the Logistic Regression classifier was exported to ONNX [5], which provides a standardized model representation that can be consumed by compatible inference runtimes without retraining. The converted model was loaded into OpenVINO [4], which can also convert it into its Intermediate Representation (IR, an XML and BIN file pair), compiled for the target device, and executed. The inference workflow is: (1) load the converted model; (2) compile it for the CPU or GPU device; (3) prepare an input batch or individual record; (4) execute the inference request; (5) obtain the predicted class; and (6) compare the predictions with those of the original scikit-learn classifier. The preprocessing pipeline must supply inputs with the expected feature ordering, data types, and encoding. Prediction agreement is measured as

$$A_{\text{agreement}} = \frac{N_{\text{matching}}}{N_{\text{compared}}} \times 100 \quad (7)$$

where N_{matching} is the number of records for which both implementations predict the same class and N_{compared} is the number of records compared.

F. Resource Inventory and Allocation Proposal

The resource module represents the availability of emergency beds, medical staff, patient monitors, and oxygen

units. The available quantity of each resource is computed from its total and occupied quantities as

$$R_{\text{available}} = R_{\text{total}} - R_{\text{occupied}} \quad (8)$$

The predicted acuity and class probabilities are displayed on the dashboard, and a rule-based module proposes beds, staff, monitors, and oxygen units for the patient. In the current implementation, inventory values are not automatically updated from predicted acuity, and no optimization algorithm has yet been validated for assigning resources to individual patients.

VI. Experimental Setup

A. Hardware and Software Environment

The experiments were performed on an Intel-based computer; the hardware and software environment is summarized in Table II.

TABLE II. Experimental Environment

Component	Specification
Hardware	Intel-based laptop/PC, minimum 8 GB RAM, integrated GPU (GPU.0)
Operating system	Windows
Language and tools	Python 3.x, VS Code, GitHub
ML libraries	NumPy, Pandas, scikit-learn, Matplotlib, joblib
Inference	ONNX, OpenVINO runtime (CPU and GPU plugins)
Optimization	SciPy, PuLP / OR-Tools
Interface	Streamlit dashboard, Flask

B. Software Installation

Python 3.x was installed and verified, and a virtual environment was created (python -m venv emergency_env) and activated. After upgrading pip, the ML libraries were installed with pip install numpy pandas scikit-learn matplotlib, followed by pip install openvino. The OpenVINO installation was verified by importing the package and printing its version. The optimization libraries (pip install ortools or pip install pulp) and Flask (pip install flask) were then installed.

C. Performance Parameters

Inference latency is the time required to generate predictions for a given input, computed as the difference between the end time and the start time of the inference call. Lower latency is desirable because timely predictions can support faster resource-planning decisions. Inference speed-up compares the baseline execution time with the OpenVINO execution time:

$$S = \frac{T_{\text{baseline}}}{T_{\text{OpenVINO}}} \quad (9)$$

For repeated measurements, the mean latency over n runs is

$$\overline{T} = \frac{1}{n}\sum_{i=1}^{n} T_i \quad (10)$$

Classification performance is measured using accuracy and balanced accuracy:

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN} \times 100 \quad (11)$$

$$BA = \frac{1}{K}\sum_{k=1}^{K} \frac{TP_k}{TP_k+FN_k} \times 100 \quad (12)$$

where TP, TN, FP, and FN denote true positives, true negatives, false positives, and false negatives, and K is the number of classes. Balanced accuracy is the average recall across classes and gives each class equal importance, which is important for the imbalanced acuity distribution. Two latency benchmarks were performed: a classifier-only benchmark that excludes preprocessing and model compilation, and an end-to-end benchmark that includes preprocessing but excludes OpenVINO model compilation. For the CPU and GPU comparison, each device executed 50 warm-up calls followed by ten timed repetitions of 1,000 inference calls.

VII. Results and Discussion

A. Classification Performance

Table III summarizes the classification results on the held-out test split. Logistic Regression achieved an accuracy of 51.92%, outperforming the Random Forest baseline accuracy of 46.15%, and it also achieved a higher balanced accuracy of 49.02% compared with 39.34%. Because the class distribution is strongly imbalanced, particularly for acuity classes 1 and 4, accuracy alone does not adequately describe model performance. The confusion matrix of the Logistic Regression model in Fig. 2 compares actual and predicted acuity levels and shows the correct and incorrect predictions for each class; no samples of acuity class 4 occurred in the test split, so performance for that class could not be evaluated. Fig. 3 compares the two models using accuracy and balanced accuracy.

TABLE III. Classification Performance

Model	Test accuracy (%)	Balanced accuracy (%)
Random Forest (baseline)	46.15	39.34
Logistic Regression	51.92	49.02



Fig. 2. Confusion matrix of the Logistic Regression model for patient acuity classification.

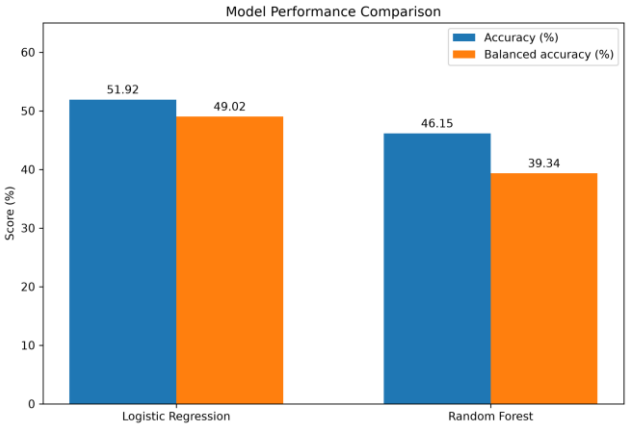


Fig. 3. Accuracy and balanced accuracy of the Logistic Regression and Random Forest models.

B. *Prediction Agreement and Classifier-Only Latency*

OpenVINO produced predictions identical to those of the original scikit-learn classifier for all 207 labelled records, giving 100% prediction agreement. Table IV reports the classifier-only benchmark. For a batch of 207 records, the scikit-learn model required 0.220828 ms per batch and OpenVINO required 0.106526 ms, a speed-up of approximately 2.07× according to (9). Separate repeated benchmarks recorded speed-ups of approximately 1.72× for batch inference and 1.83× for single-record inference.

TABLE IV. PREDICTION AGREEMENT AND CLASSIFIER-ONLY BENCHMARK

Measurement	Result
Prediction agreement (207 records)	100%
scikit-learn time per batch	0.220828 ms
OpenVINO CPU time per batch	0.106526 ms
Speed-up, batch of 207 records	≈ 2.07×
Speed-up, repeated batch benchmark	≈ 1.72×
Speed-up, single-record benchmark	≈ 1.83×

C. *End-to-End Latency*

Table V reports the end-to-end latency, which includes preprocessing. In the first two runs the OpenVINO pipeline was slower than the scikit-learn pipeline, and in the third run the two were almost identical (a speed ratio of approximately 1.01×). These results show that preprocessing and execution overhead can reduce or eliminate the classifier-only gain, and that classifier-only and complete-pipeline performance must be evaluated separately.

TABLE V. END-TO-END PIPELINE LATENCY

Run	scikit-learn latency (ms)	OpenVINO pipeline mean latency (ms)
1	6.551330	7.914014
2	6.305257	6.651685
3	5.444239	5.416421

D. *CPU Versus GPU Inference*

The model was compiled and executed successfully on both the CPU and GPU.0 using the OpenVINO CPU and GPU plugins, and a technical test with an input tensor of shape (1, 168) confirmed GPU inference. As shown in Table VI, the CPU recorded a mean latency of 0.168683 ms and a median of 0.167199 ms, while the GPU recorded a mean latency of 0.543031 ms and a median of 0.391393 ms. Based on mean latency, the CPU was approximately 3.22 times faster than the GPU, suggesting that GPU execution overhead outweighs the benefits of parallel processing for this small Logistic Regression model. Figs. 4 and 5 report a separate benchmark of 50 runs of 1,000 inferences each: Fig. 4 compares the average latency (approximately 0.113 ms on the CPU and 0.369 ms on the GPU), and Fig. 5 shows the latency of every run, highlighting that CPU execution was also more consistent. CPU execution is therefore preferred for the tested single-input workload.

TABLE VI. OPENVINO CPU AND GPU INFERENCE LATENCY

Parameter	CPU	GPU.0
Model execution	Successful	Successful
Mean latency	0.168683 ms	0.543031 ms
Median latency	0.167199 ms	0.391393 ms
Minimum latency	0.123318 ms	—
Maximum latency	0.261083 ms	0.900240 ms
Standard deviation	0.038313 ms	0.251192 ms
Timed repetitions	10	10
Timed calls per repetition	1000	1000

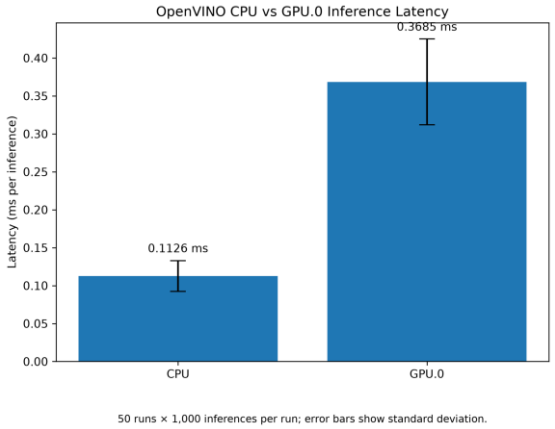


Fig. 4. Average OpenVINO inference latency on CPU and GPU.0 (50 runs × 1,000 inferences; error bars show standard deviation).

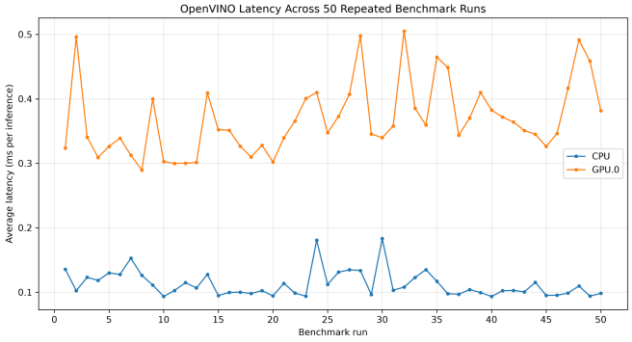


Fig. 5. OpenVINO inference latency across 50 repeated benchmark runs on CPU and GPU.0.

E. *Resource Availability and Decision Support*

Fig. 6 illustrates the total, occupied, and available quantities of emergency beds, medical staff, patient monitors, and oxygen units, providing an overview of resource utilization for emergency planning. The predicted acuity, together with the current resource inventory, can support prioritization by helping identify emergency cases and available resources. Figs. 7–11 show the Streamlit interface: the main dashboard with the current resource inventory (Fig. 7), the patient assessment form for vital signs and patient information (Fig. 8), the OpenVINO prediction output with class probabilities (Fig. 9), the resource allocation proposal (Fig. 10), and the patient allocation records with remaining

resources (Fig. 11). All prioritization and allocation decisions require verification by qualified healthcare personnel.

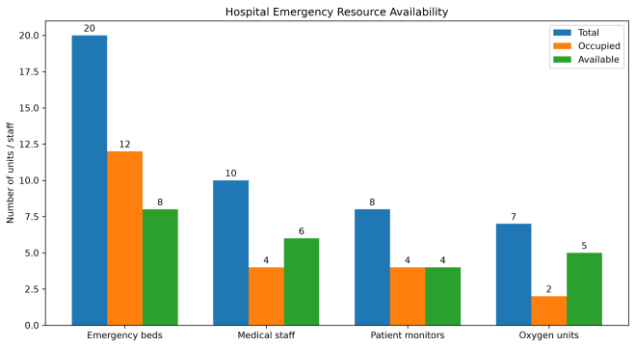


Fig. 6. Total, occupied, and available hospital emergency resources.

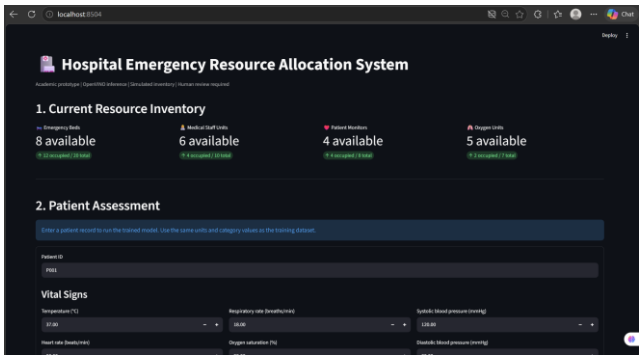


Fig. 7. Main dashboard of the Hospital Emergency Resource Allocation System showing the current resource inventory.

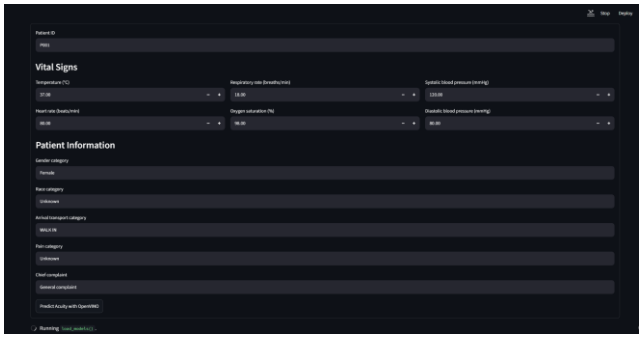


Fig. 8. Patient assessment interface for entering vital signs and patient information.

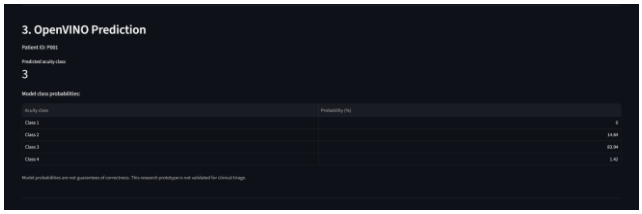


Fig. 9. OpenVINO prediction interface showing the predicted acuity and class probabilities.

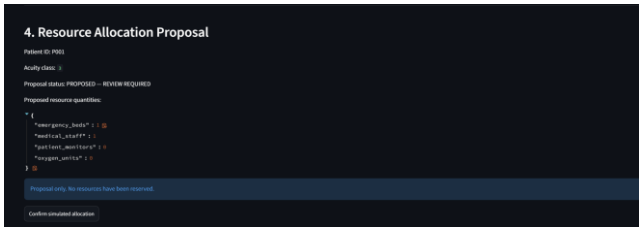


Fig. 10. Resource allocation proposal generated for the assessed patient.

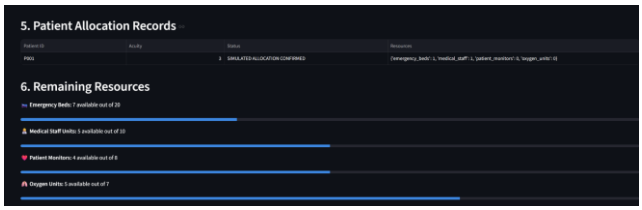


Fig. 11. Patient allocation records and remaining resources.

F. Limitations

The current system has limitations arising from the size and class distribution of the dataset, which may affect the reliability of acuity predictions. The test split contained no samples of acuity class 4, so performance for that class could not be evaluated. The CPU and GPU benchmarks measure model execution under a controlled input rather than the complete hospital workflow, and the resource proposals are rule-based and have not been validated against clinical outcomes. Further validation using larger and more representative datasets, such as the full MIMIC-IV-ED database, is required before considering real-world clinical use.

VIII. Conclusion and Future Work

This paper presented a prototype Hospital Emergency Resource Allocation System that combines ML-based patient acuity classification, OpenVINO inference, and hospital resource availability monitoring. Logistic Regression achieved 51.92% accuracy compared with 46.15% for the Random Forest baseline, the OpenVINO-executed model reproduced the scikit-learn predictions on all 207 labelled records, and classifier-only inference was up to approximately 2.07× faster with OpenVINO. However, end-to-end measurements showed that preprocessing overhead can remove this gain, and CPU execution was approximately 3.22 times faster than GPU execution for this small model. Although the results demonstrate the feasibility of integrating these components into a single prototype, improvements in model accuracy, dataset representation, and system-level validation are needed before practical clinical deployment. Future work will validate resource-demand predictions on larger and independent datasets, address class imbalance, add explainability using feature importance or SHAP, evaluate optimization-based allocation strategies using PuLP or OR-Tools, and test performance on additional hardware.

References

[1] A. Zlotnik, A. Gallardo-Antolín, M. Cuchí Alfaro, M. C. Pérez Pérez, and J. M. Montero Martínez, “Emergency department visit forecasting and dynamic nursing staff allocation using machine learning techniques with readily available open-source software,” *Comput. Inform. Nurs.*, vol. 33, no. 8, pp. 368–377, 2015, doi: 10.1097/CIN.0000000000000173.

[2] T. Mizan and S. Taghipour, “Medical resource allocation planning by integrating machine learning and optimization models,” *Artif. Intell. Med.*, vol. 134, Art. no. 102430, 2022, doi: 10.1016/j.artmed.2022.102430.

[3] A. B. El Ariss *et al.*, “Development and validation of a machine learning framework for improved resource allocation in the emergency department,” *Am. J. Emerg. Med.*, vol. 84, pp. 141–148, 2024, doi: 10.1016/j.ajem.2024.07.040.

[4] Intel Corporation, “OpenVINO toolkit documentation.” [Online]. Available: https://docs.openvino.ai

[5] ONNX, “Open Neural Network Exchange.” [Online]. Available: https://onnx.ai

[6] M. Xu, T. C. Wong, and K. S. Chin, “Modeling daily patient arrivals at emergency department and quantifying the relative importance of contributing variables using artificial neural network,” *Decis. Support Syst.*, vol. 54, no. 3, pp. 1488–1498, 2013, doi: 10.1016/j.dss.2012.12.019.

[7] T.-L. Chen and C.-C. Wang, “Multi-objective simulation optimization for medical capacity allocation in emergency department,” *J. Simul.*, vol. 10, no. 1, pp. 50–68, 2016, doi: 10.1057/jos.2014.39.

[8] M. Yousefi, M. Yousefi, R. P. M. Ferreira, J. H. Kim, and F. S. Fogliatto, “Chaotic genetic algorithm and Adaboost ensemble metamodeling approach for optimum resource planning in emergency departments,” *Artif. Intell. Med.*, vol. 84, pp. 23–33, 2018, doi: 10.1016/j.artmed.2017.10.002.

[9] W. S. Hong, A. D. Haimovich, and R. A. Taylor, “Predicting hospital admission at emergency department triage using machine learning,” *PLoS ONE*, vol. 13, no. 7, Art. no. e0201016, 2018, doi: 10.1371/journal.pone.0201016.

[10] Y. Raita, T. Goto, M. K. Faridi, D. F. M. Brown, C. A. Camargo Jr., and K. Hasegawa, “Emergency department triage prediction of clinical outcomes using machine learning models,” *Crit. Care*, vol. 23, Art. no. 64, 2019, doi: 10.1186/s13054-019-2351-7.

[11] F. Harrou, A. Dairi, F. Kadri, and Y. Sun, “Forecasting emergency department overcrowding: A deep learning framework,” *Chaos Solitons Fractals*, vol. 139, Art. no. 110247, 2020, doi: 10.1016/j.chaos.2020.110247.

[12] N. W. Sterling *et al.*, “Prediction of emergency department resource requirements during triage: An application of current natural language processing techniques,” *J. Am. Coll. Emerg. Physicians Open*, vol. 1, no. 6, pp. 1676–1683, 2020, doi: 10.1002/emp2.12253.

[13] E. Benevento, D. Aloini, and N. Squicciarini, “Towards a real-time prediction of waiting times in emergency departments: A comparative analysis of machine learning techniques,” *Int. J. Forecast.*, vol. 39, no. 1, pp. 192–208, 2023, doi: 10.1016/j.ijforecast.2021.10.006.

[14] J. W. Joseph *et al.*, “Machine learning methods for predicting patient-level emergency department workload,” *J. Emerg. Med.*, vol. 64, no. 1, pp. 83–92, 2023, doi: 10.1016/j.jemermed.2022.10.002.

[15] A. J. Smith *et al.*, “Multisite evaluation of prediction models for emergency department crowding before and during the COVID-19 pandemic,” *J. Am. Med. Inform. Assoc.*, vol. 30, no. 2, pp. 292–300, 2023, doi: 10.1093/jamia/ocac214.

[16] M. A. C. Vollmer *et al.*, “A unified machine learning approach to time series forecasting applied to demand at emergency departments,” *BMC Emerg. Med.*, vol. 21, Art. no. 9, 2021, doi: 10.1186/s12873-020-00395-y.

[17] C. N. Rocha and F. Rodrigues, “Forecasting emergency department admissions,” *J. Intell. Inf. Syst.*, vol. 56, no. 3, pp. 509–528, 2021, doi: 10.1007/s10844-021-00638-9.

[18] J. Tuominen *et al.*, “Forecasting emergency department occupancy with advanced machine learning models and multivariable input,” *Int. J. Forecast.*, vol. 40, no. 4, pp. 1410–1420, 2024, doi: 10.1016/j.ijforecast.2023.12.002.

[19] B. M. Porto, “Improving triage performance in emergency departments using machine learning and natural language processing: A systematic review,” *BMC Emerg. Med.*, vol. 24, Art. no. 219, 2024, doi: 10.1186/s12873-024-01135-2.

[20] B. M. Porto and F. S. Fogliatto, “Enhanced forecasting of emergency department patient arrivals using feature engineering approach and machine learning,” *BMC Med. Inform. Decis. Mak.*, vol. 24, Art. no. 377, 2024, doi: 10.1186/s12911-024-02788-6.

[21] J. Blanco, M. Ferreras, and O. Cosido, “Predictive modeling of hospital emergency department demand using artificial intelligence: A systematic review,” *Int. J. Med. Inform.*, vol. 207, Art. no. 106215, 2026, doi: 10.1016/j.ijmedinf.2025.106215.

[22] E. E. Schippers, S. P. Willemsen, L. A. M. Collée, K. L. M. Koenraadt, and S. Schol-Gelok, “Crowding in real-time: An artificial intelligence-based model to manage crowding in the emergency department (CRAMMED study),” *BMC Emerg. Med.*, vol. 26, Art. no. 232, 2026, doi: 10.1186/s12873-026-01647-z.

[23] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[24] A. Johnson, L. Bulgarelli, T. Pollard, L. A. Celi, R. Mark, and S. Horng, “MIMIC-IV-ED (version 2.2),” PhysioNet, 2023, doi: 10.13026/5ntk-km72.

[25] L. Breiman, “Random forests,” *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

https://github.com/nisharadhika/hospital-emergency-resource-allocation-openvino